

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 05

NOMBRE COMPLETO: Pablo Sebastian Noyola Torres

Nº de Cuenta: 320023213

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027 - 1

FECHA DE ENTREGA LÍMITE: 26 de septiembre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Importar el modelo del Holocron con jerarquía

Capturas de código:

```
61    Model HolocronCuerpo;
62    Model HolocronPiramide1;
63    Model HolocronPiramide2;
64    Model HolocronPiramide3;
65    Model HolocronPiramide4;
66    Model HolocronPiramide5;
67    Model HolocronPiramide6;
68    Model HolocronPiramide7;
69    Model HolocronPiramide8;
70
```

```
197     HolocronCuerpo = Model();
198     HolocronCuerpo.LoadModel("Models/HolocronCuerpo.obj");
199     HolocronPiramide1 = Model();
200     HolocronPiramide1.LoadModel("Models/HolocronPiramide1.obj");
201     HolocronPiramide2 = Model();
202     HolocronPiramide2.LoadModel("Models/HolocronPiramide2.obj");
203     HolocronPiramide3 = Model();
204     HolocronPiramide3.LoadModel("Models/HolocronPiramide3.obj");
205     HolocronPiramide4 = Model();
206     HolocronPiramide4.LoadModel("Models/HolocronPiramide4.obj");
207     HolocronPiramide5 = Model();
208     HolocronPiramide5.LoadModel("Models/HolocronPiramide5.obj");
209     HolocronPiramide6 = Model();
210     HolocronPiramide6.LoadModel("Models/HolocronPiramide6.obj");
211     HolocronPiramide7 = Model();
212     HolocronPiramide7.LoadModel("Models/HolocronPiramide7.obj");
213     HolocronPiramide8 = Model();
214     HolocronPiramide8.LoadModel("Models/HolocronPiramide8.obj");
```

```
        float rotH = mainWindow.getAnguloHolocron();
        float rotP = rotH * 2.0f;
        float d = mainWindow.getDistanciaPiramides();

        // Cuerpo principal del Holocron
        color = glm::vec3(0.10f, 0.18f, 0.29f);
        modelCuerpoH = glm::mat4(1.0);
        modelCuerpoH = glm::translate(modelCuerpoH, glm::vec3(30.0f, 10.0f, 0.0f));
        modelCuerpoH = glm::rotate(modelCuerpoH, glm::radians(rotH), glm::vec3(0.0f, 1.0f, 0.0f));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelCuerpoH));
        HolocronCuerpo.RenderModel();

        // Piramide 1
        color = glm::vec3(0.83f, 0.69f, 0.22f);
        glm::mat4 modelPiramide1 = modelCuerpoH;
        modelPiramide1 = glm::translate(modelPiramide1, glm::vec3(3.37f * d, 3.35f * d, 3.6f * d));
        modelPiramide1 = glm::rotate(modelPiramide1, glm::radians(rotP + mainWindow.getarticulacion20()), glm::vec3(1.0f, 1.0f, 1.0f));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPiramide1));
        HolocronPiramide1.RenderModel();

        // Piramide 2
        color = glm::vec3(0.83f, 0.69f, 0.22f);
        glm::mat4 modelPiramide2 = modelCuerpoH;
        modelPiramide2 = glm::translate(modelPiramide2, glm::vec3(3.51f * d, 3.6f * d, -3.27f * d));
        modelPiramide2 = glm::rotate(modelPiramide2, glm::radians(rotP + mainWindow.getarticulacion21()), glm::vec3(1.0f, 1.0f, -1.0f));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPiramide2));
        HolocronPiramide2.RenderModel();
```

```
        HolocronPiramide2.RenderModel();

        // Piramide 3
        color = glm::vec3(0.83f, 0.69f, 0.22f);
        glm::mat4 modelPiramide3 = modelCuerpoH;
        modelPiramide3 = glm::translate(modelPiramide3, glm::vec3(-3.5f * d, 3.5f * d, 3.94f * d));
        modelPiramide3 = glm::rotate(modelPiramide3, glm::radians(rotP + mainWindow.getarticulacion22()), glm::vec3(-1.0f, 1.0f, 1.0f));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPiramide3));
        HolocronPiramide3.RenderModel();

        // Piramide 4
        color = glm::vec3(0.83f, 0.69f, 0.22f);
        glm::mat4 modelPiramide4 = modelCuerpoH;
        modelPiramide4 = glm::translate(modelPiramide4, glm::vec3(-3.68f * d, 3.45f * d, -3.23f * d));
        modelPiramide4 = glm::rotate(modelPiramide4, glm::radians(rotP + mainWindow.getarticulacion23()), glm::vec3(-1.0f, 1.0f, -1.0f));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPiramide4));
        HolocronPiramide4.RenderModel();

        // Piramide 5
        color = glm::vec3(0.83f, 0.69f, 0.22f);
        glm::mat4 modelPiramide5 = modelCuerpoH;
        modelPiramide5 = glm::translate(modelPiramide5, glm::vec3(3.33f * d, -3.55f * d, 3.4f * d));
        modelPiramide5 = glm::rotate(modelPiramide5, glm::radians(rotP + mainWindow.getarticulacion24()), glm::vec3(1.0f, -1.0f, 1.0f));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPiramide5));
        HolocronPiramide5.RenderModel();

        // Piramide 6
        color = glm::vec3(0.83f, 0.69f, 0.22f);
        glm::mat4 modelPiramide6 = modelCuerpoH;
        modelPiramide6 = glm::translate(modelPiramide6, glm::vec3(3.41f * d, -3.64f * d, -3.59f * d));
        modelPiramide6 = glm::rotate(modelPiramide6, glm::radians(rotP + mainWindow.getarticulacion25()), glm::vec3(1.0f, -1.0f, -1.0f));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
```

```
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPiramide6));
        HolocronPiramide6.RenderModel();

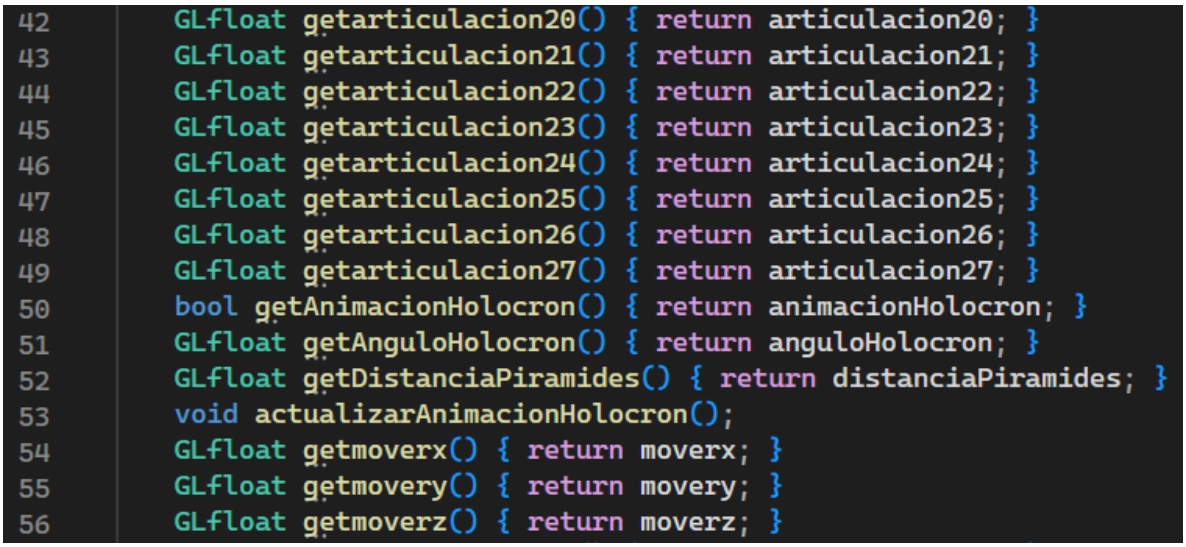
        // Piramide 7
        color = glm::vec3(0.83f, 0.69f, 0.22f);
        glm::mat4 modelPiramide7 = modelCuerpoH;
        modelPiramide7 = glm::translate(modelPiramide7, glm::vec3(-3.78f * d, -3.43f * d, -3.29f * d));
        modelPiramide7 = glm::rotate(modelPiramide7, glm::radians(rotP + mainWindow.getarticulacion26()), glm::vec3(-1.0f, -1.0f, -1.0f));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPiramide7));
        HolocronPiramide7.RenderModel();

        // Piramide 8
        color = glm::vec3(0.83f, 0.69f, 0.22f);
        glm::mat4 modelPiramide8 = modelCuerpoH;
        modelPiramide8 = glm::translate(modelPiramide8, glm::vec3(-3.75f * d, -3.62f * d, 4.83f * d));
        modelPiramide8 = glm::rotate(modelPiramide8, glm::radians(rotP + mainWindow.getarticulacion27()), glm::vec3(-1.0f, -1.0f, 1.0f));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPiramide8));
        HolocronPiramide8.RenderModel();
        // ------------------------------------------------------------------------------------------------------------------------------------------
```

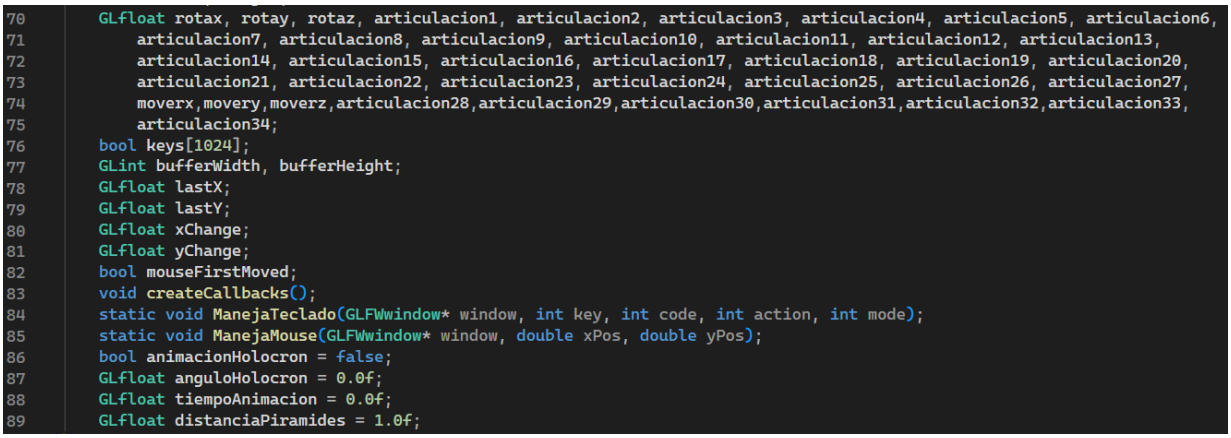
Para implementar el modelo jerárquico del Holocron, primero se realizo la declaración de los objetos de tipo Model correspondientes al cuerpo central y a las ocho pirámides, los cuales se inicializan cargando sus respectivos archivos .obj desde el directorio de recursos. En la fase de renderizado, se define el cuerpo principal del Holocron (HolocronCuerpo) utilizando una matriz base con transformaciones de traslación y rotación global controladas por teclado. A partir de esta matriz base, las ocho pirámides se heredan de manera jerárquica asignándoles modelCuerpoH, aplicando traslaciones relativas escaladas por una variable de distancia, y sumando rotaciones tanto colectivas como independientes mediante funciones de articulación específicas para cada componente exterior.

2.- Con teclado de forma independiente poder girar cada esquina alrededor del centro del Holocrón.

Capturas de código:



```
42      GLfloat getarticulacion20() { return articulacion20; }
43      GLfloat getarticulacion21() { return articulacion21; }
44      GLfloat getarticulacion22() { return articulacion22; }
45      GLfloat getarticulacion23() { return articulacion23; }
46      GLfloat getarticulacion24() { return articulacion24; }
47      GLfloat getarticulacion25() { return articulacion25; }
48      GLfloat getarticulacion26() { return articulacion26; }
49      GLfloat getarticulacion27() { return articulacion27; }
50      bool getAnimacionHolocron() { return animacionHolocron; }
51      GLfloat getAnguloHolocron() { return anguloHolocron; }
52      GLfloat getDistanciaPiramides() { return distanciaPiramides; }
53      void actualizarAnimacionHolocron();
54      GLfloat getmoverx() { return moverx; }
55      GLfloat getmovery() { return movery; }
56      GLfloat getmoverz() { return moverz; }
```



```
70      GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6,
71          articulacion7, articulacion8, articulacion9, articulacion10, articulacion11, articulacion12, articulacion13,
72          articulacion14, articulacion15, articulacion16, articulacion17, articulacion18, articulacion19, articulacion20,
73          articulacion21, articulacion22, articulacion23, articulacion24, articulacion25, articulacion26, articulacion27,
74          moverx,movery,moverz,articulacion28,articulacion29,articulacion30,articulacion31,articulacion32,articulacion33,
75          articulacion34;
76      bool keys[1024];
77      GLint bufferWidth, bufferHeight;
78      GLfloat lastX;
79      GLfloat lastY;
80      GLfloat xChange;
81      GLfloat yChange;
82      bool mouseFirstMoved;
83      void createCallbacks();
84      static void ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode);
85      static void ManejaMouse(GLFWwindow* window, double xPos, double yPos);
86      bool animacionHolocron = false;
87      GLfloat anguloHolocron = 0.0f;
88      GLfloat tiempoAnimacion = 0.0f;
89      GLfloat distanciaPiramides = 1.0f;
```

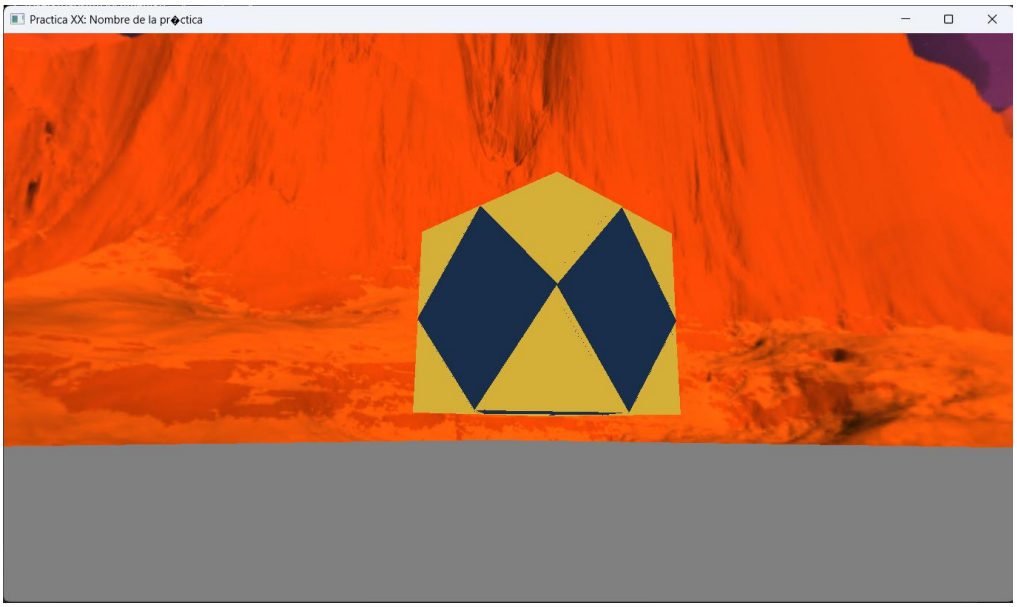
```
39      articulacion20 = 0.0f;
40      articulacion21 = 0.0f;
41      articulacion22 = 0.0f;
42      articulacion23 = 0.0f;
43      articulacion24 = 0.0f;
44      articulacion25 = 0.0f;
45      articulacion26 = 0.0f;
46      articulacion27 = 0.0f;
47      moverx = 0.0f;
48      movery = 0.0f;
49      moverz = 0.0f;
```

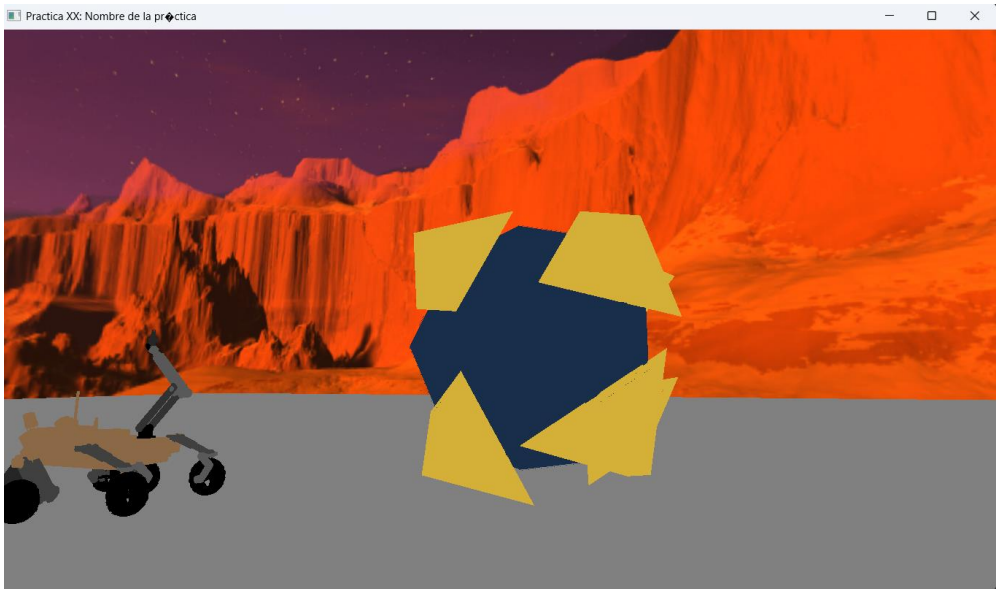
```
368     // Letras con Shift
369
370     if (key == GLFW_KEY_1 && (mode & GLFW_MOD_SHIFT))
371     {
372         theWindow->articulacion20 += 10.0f;
373     }
374
375     if (key == GLFW_KEY_2 && (mode & GLFW_MOD_SHIFT))
376     {
377         theWindow->articulacion21 += 10.0f;
378     }
379
380     if (key == GLFW_KEY_3 && (mode & GLFW_MOD_SHIFT))
381     {
382         theWindow->articulacion22 += 10.0f;
383     }
384
385     if (key == GLFW_KEY_4 && (mode & GLFW_MOD_SHIFT))
386     {
387         theWindow->articulacion23 += 10.0f;
388     }
389
390     if (key == GLFW_KEY_5 && (mode & GLFW_MOD_SHIFT))
391     {
392         theWindow->articulacion24 += 10.0f;
393     }
394
395     if (key == GLFW_KEY_6 && (mode & GLFW_MOD_SHIFT))
396     {
397         theWindow->articulacion25 += 10.0f;
398     }
399
400     if (key == GLFW_KEY_7 && (mode & GLFW_MOD_SHIFT))
```

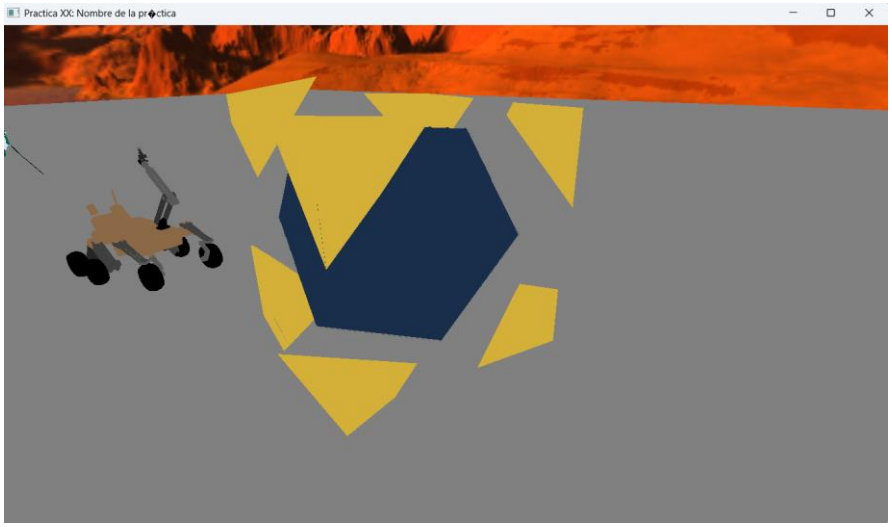
```
400         if (key == GLFW_KEY_7 && (mode & GLFW_MOD_SHIFT))
401         {
402             theWindow->articulacion26 += 10.0f;
403         }
404
405         if (key == GLFW_KEY_8 && (mode & GLFW_MOD_SHIFT))
406         {
407             theWindow->articulacion27 += 10.0f;
408         }
409
410         if (key == GLFW_KEY_9 && (mode & GLFW_MOD_SHIFT) && action == GLFW_PRESS)
411         {
412             theWindow->animacionHolocron = !theWindow->animacionHolocron;
413         }
414
415         if (key == GLFW_KEY_D && action == GLFW_PRESS)
416         {
417             //const char* key_name = glfwGetKeyName(GLFW_KEY_D, 0);
418         }
```

```
475 void Window::actualizarAnimacionHolocron()
476 {
477     if (animacionHolocron)
478     {
479         anguloHolocron += 0.03f;
480         if (anguloHolocron > 360.0f) {
481             anguloHolocron -= 360.0f;
482         }
483
484         tiempoAnimacion += 0.001f;
485         distanciaPiramides = 1.0f + (pow(sin(tiempoAnimacion), 2.0f) * 0.6f);
486     }
487     else
488     {
489         if (tiempoAnimacion > 0.0f) {
490             tiempoAnimacion += 0.005f;
491             if (sin(tiempoAnimacion) <= 0.01f) {
492                 tiempoAnimacion = 0.0f;
493                 distanciaPiramides = 1.0f;
494             }
495             else {
496                 distanciaPiramides = 1.0f + (pow(sin(tiempoAnimacion), 2.0f) * 0.6f);
497             }
498         }
499     }
500 }
```

Capturas de ejecución:


Practica XX: Nombre de la pr&#65533;ctica


Practica XX: Nombre de la pr&#65533;ctica



Video de ejecución: https://www.youtube.com/watch?v=jNYHoSSPD3s

Para lograr el control independiente de cada esquina alrededor del centro del Holocrón, se amplió la clase Window (Window.h y Window.cpp) incorporando variables de articulación del 20 al 27 junto con sus respectivos métodos de acceso. En la función de manejo de eventos de teclado, se configuraron entradas específicas utilizando combinaciones con la tecla Shift (GLFW_KEY_1 hasta GLFW_KEY_8) que permiten incrementar en 10° el valor de cada articulación correspondiente. Asimismo, se añadió una opción de animación automatizada (animacionHolocron) que gestiona de forma fluida el ángulo de rotación global y la distancia de expansión de las pirámides mediante funciones trigonométricas en la función actualizarAnimacionHolocron(). Como resultado en la ejecución gráfica, el modelo tridimensional permite visualizar tanto el estado cerrado e integrado del Holocrón como la apertura dinámica y la rotación independiente de sus esquinas poligonales alrededor del núcleo central.

3.- Importar el satélite con jerarquía

Capturas de código:

```
71    Model BaseSatelite;
72    Model AntenaSuperior;
73    Model Varilla;
74    Model Panel1;
75    Model Panel2;
```

```
575     // Satelite ----------------------------------------------------------------------------------------------------------
576
577     // 1. Cuerpo principal del Satelite
578     color = glm::vec3(0.85f, 0.88f, 0.92f);
579     modelCuerpoS = glm::mat4(1.0);
580     modelCuerpoS = glm::translate(modelCuerpoS, glm::vec3(-30.0f + mainWindow.getmoverx(), 10.0f + mainWindow.getmovery(), 0.0f + mainWindow.getmoverz()));
581     modelCuerpoS = glm::rotate(modelCuerpoS, glm::radians(mainWindow.getarticulacion28()), glm::vec3(1.0f, 0.0f, 0.0f));
582     modelCuerpoS = glm::rotate(modelCuerpoS, glm::radians(mainWindow.getarticulacion29()), glm::vec3(0.0f, 1.0f, 0.0f));
583     modelCuerpoS = glm::rotate(modelCuerpoS, glm::radians(mainWindow.getarticulacion30()), glm::vec3(0.0f, 0.0f, 1.0f));
584     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
585     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelCuerpoS));
586     BaseSatelite.RenderModel();
587
588
589     // Panel1
590     color = glm::vec3(0.0, 0.408, 0.278);
591     glm::mat4 modelPanel1 = modelCuerpoS;
592     modelPanel1 = glm::translate(modelPanel1, glm::vec3(-0.82f, 0.16f, -0.43f));
593     modelPanel1 = glm::rotate(modelPanel1, glm::radians(mainWindow.getarticulacion31()), glm::vec3(1.0f, 0.0f, 0.0f));
594     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
595     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPanel1));
596     Panel1.RenderModel();
597
598     // Panel2
599     color = glm::vec3(0.0, 0.408, 0.278);
600     glm::mat4 modelPanel2 = modelCuerpoS;
601     modelPanel2 = glm::translate(modelPanel2, glm::vec3(0.82f, 0.16f, -0.43f));
602     modelPanel2 = glm::rotate(modelPanel2, glm::radians(mainWindow.getarticulacion32()), glm::vec3(1.0f, 0.0f, 0.0f));
603     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
604     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPanel2));
605     Panel2.RenderModel();
```

```
598     // Panel2
599     color = glm::vec3(0.0, 0.408, 0.278);
600     glm::mat4 modelPanel2 = modelCuerpoS;
601     modelPanel2 = glm::translate(modelPanel2, glm::vec3(0.82f, 0.16f, -0.43f));
602     modelPanel2 = glm::rotate(modelPanel2, glm::radians(mainWindow.getarticulacion32()), glm::vec3(1.0f, 0.0f, 0.0f));
603     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
604     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPanel2));
605     Panel2.RenderModel();
606
607
608     // Varilla
609     color = glm::vec3(0.0, 0.0, 0.0);
610     glm::mat4 modelVarilla = modelCuerpoS;
611     modelVarilla = glm::translate(modelVarilla, glm::vec3(0.33f, -0.42f, -1.23f));
612     modelVarilla = glm::rotate(modelVarilla, glm::radians(mainWindow.getarticulacion33()), glm::vec3(0.0f, 1.0f, 0.0f));
613     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
614     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelVarilla));
615     Varilla.RenderModel();
616
617     // Antena Superior
618     color = glm::vec3(0.0, 0.0, 0.0);
619     glm::mat4 modelAntena = modelCuerpoS;
620     modelAntena = glm::translate(modelAntena, glm::vec3(0.0f, 1.06f, 0.0f));
621     modelAntena = glm::rotate(modelAntena, glm::radians(mainWindow.getarticulacion34()), glm::vec3(0.0f, 1.0f, 0.0f));
622     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
623     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelAntena));
624     AntenaSuperior.RenderModel();
```

Para estructurar el modelo jerárquico del satélite, primero se declararon e inicializaron los objetos de tipo Model correspondientes a cada una de sus partes como la base central, la antena superior, la varilla y los paneles solares cargando sus respectivos archivos .obj desde los recursos del proyecto. En el proceso de renderizado, se definió la matriz base del cuerpo principal (modelCuerpoS), aplicando traslaciones y rotaciones globales que permiten ubicar la base del satélite en la escena. A partir de esta matriz principal, las demás estructuras (como Panel1, Panel2, la varilla y la antena superior) heredan la posición de forma jerárquica asignando modelCuerpoS como su matriz padre, lo que permite aplicarles desplazamientos locales y rotaciones independientes manteniendo la coherencia espacial de todo el conjunto.

4.- Agregar movimiento al satélite con teclado sobre los ejes X, Y, Z

Capturas de código:

```
57      GLfloat getarticulacion28() { return articulacion28; }
58      GLfloat getarticulacion29() { return articulacion29; }
59      GLfloat getarticulacion30() { return articulacion30; }
60      GLfloat getarticulacion31() { return articulacion31; }
61      GLfloat getarticulacion32() { return articulacion32; }
62      GLfloat getarticulacion33() { return articulacion33; }
63      GLfloat getarticulacion34() { return articulacion34; }
64
65
66      ~Window();
67  private:
68      GLFWwindow *mainWindow;
69      GLint width, height;
70      GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6,
71          articulacion7, articulacion8, articulacion9, articulacion10, articulacion11, articulacion12, articulacion13,
72          articulacion14, articulacion15, articulacion16, articulacion17, articulacion18, articulacion19, articulacion20,
73          articulacion21, articulacion22, articulacion23, articulacion24, articulacion25, articulacion26, articulacion27,
74          moverx,movery,moverz,articulacion28,articulacion29,articulacion30,articulacion31,articulacion32,articulacion33,
75          articulacion34;
```

```
50      articulacion28 = 0.0f;
51      articulacion29 = 0.0f;
52      articulacion30 = 0.0f;
53      articulacion31 = 0.0f;
54      articulacion32 = 0.0f;
55      articulacion33 = 0.0f;
56      articulacion34 = 0.0f;
```

```
148     // ROTACIONES / ARTICULACIONES CON E, R, T
149     if (key == GLFW_KEY_E && !(mode & GLFW_MOD_SHIFT))
150     {
151         theWindow->articulacion28 += 10.0f;
152     }
153
154     if (key == GLFW_KEY_R && !(mode & GLFW_MOD_SHIFT))
155     {
156         theWindow->articulacion29 += 10.0f;
157     }
158
159     if (key == GLFW_KEY_T && !(mode & GLFW_MOD_SHIFT))
160     {
161         theWindow->articulacion30 += 10.0f;
162     }
```

```
452     if (key == GLFW_KEY_UP && !(mode & GLFW_MOD_SHIFT))
453     {
454         theWindow->articulacion31 += 10.0f;
455     }
456
457     if (key == GLFW_KEY_DOWN && !(mode & GLFW_MOD_SHIFT))
458     {
459         theWindow->articulacion32 += 10.0f;
460     }
461
462     if (key == GLFW_KEY_LEFT && !(mode & GLFW_MOD_SHIFT))
463     {
464         theWindow->articulacion33 += 10.0f;
465     }
466
467     if (key == GLFW_KEY_RIGHT && !(mode & GLFW_MOD_SHIFT))
468     {
469         theWindow->articulacion34 += 10.0f;
470     }
```

Para otorgar interactividad y movimiento al satélite sobre los ejes X, Y y Z en tiempo real, se ampliaron las clases Window.h y Window.cpp incorporando las variables de articulación del 28 al 34 junto con sus respectivos métodos de acceso. En la función de manejo de eventos de teclado, se configuraron entradas específicas mediante las teclas E, R, T y las teclas de dirección (UP, DOWN, LEFT, RIGHT) que permiten incrementar en 10° el valor de las rotaciones y desplazamientos correspondientes del satélite. De este modo, al presionar las teclas asignadas, el sistema actualiza los parámetros en tiempo real, permitiendo controlar con precisión la orientación y posición espacial de todo el conjunto jerárquico dentro de la escena gráfica.

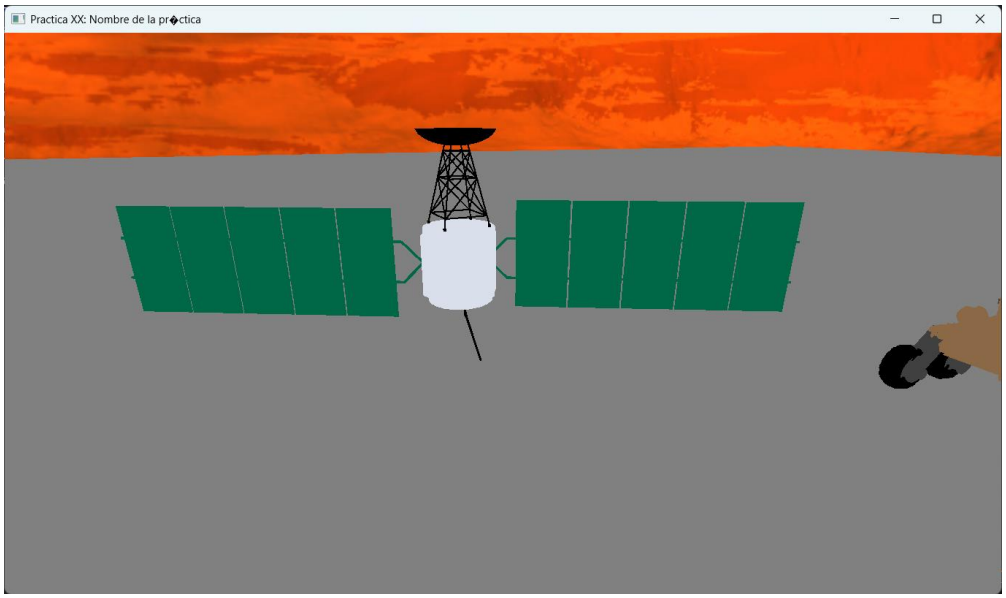
5.- Agregar rotaciones a por lo menos 3 partes del satélite que tengan contexto adecuado. Por ejemplo, si el satélite tiene un panel de celdas solares, solo debería de girar en la conexión con el cuerpo, no alrededor de todo el cuerpo.

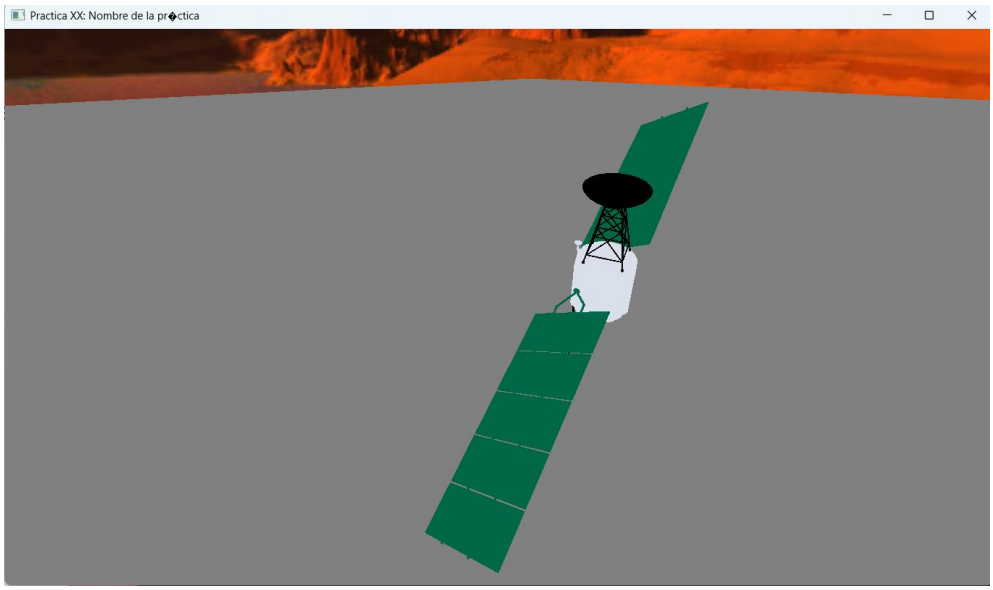
Capturas de código:

```
577         // 1. Cuerpo principal del Satelite
578         color = glm::vec3(0.85f, 0.88f, 0.92f);
579         modelCuerpoS = glm::mat4(1.0);
580         modelCuerpoS = glm::translate(modelCuerpoS, glm::vec3(-30.0f + mainWindow.getmoverx(), 10.0f + mainWindow.getmovery(), 0.0f + mainWindow.getmoverz()));
581         modelCuerpoS = glm::rotate(modelCuerpoS, glm::radians(mainWindow.getarticulacion28()), glm::vec3(1.0f, 0.0f, 0.0f));
582         modelCuerpoS = glm::rotate(modelCuerpoS, glm::radians(mainWindow.getarticulacion29()), glm::vec3(0.0f, 1.0f, 0.0f));
583         modelCuerpoS = glm::rotate(modelCuerpoS, glm::radians(mainWindow.getarticulacion30()), glm::vec3(0.0f, 0.0f, 1.0f));
584         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
585         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelCuerpoS));
586         BaseSatelite.RenderModel();
587
588
589         // Panel1
590         color = glm::vec3(0.0, 0.408, 0.278);
591         glm::mat4 modelPanel1 = modelCuerpoS;
592         modelPanel1 = glm::translate(modelPanel1, glm::vec3(-0.82f, 0.16f, -0.43f));
593         modelPanel1 = glm::rotate(modelPanel1, glm::radians(mainWindow.getarticulacion31()), glm::vec3(1.0f, 0.0f, 0.0f));
594         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
595         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPanel1));
596         Panel1.RenderModel();
597
598         // Panel2
599         color = glm::vec3(0.0, 0.408, 0.278);
600         glm::mat4 modelPanel2 = modelCuerpoS;
601         modelPanel2 = glm::translate(modelPanel2, glm::vec3(0.82f, 0.16f, -0.43f));
602         modelPanel2 = glm::rotate(modelPanel2, glm::radians(mainWindow.getarticulacion32()), glm::vec3(1.0f, 0.0f, 0.0f));
603         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
604         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPanel2));
605         Panel2.RenderModel();
```

```
608         // Varilla
609         color = glm::vec3(0.0, 0.0, 0.0);
610         glm::mat4 modelVarilla = modelCuerpoS;
611         modelVarilla = glm::translate(modelVarilla, glm::vec3(0.33f, -0.42f, -1.23f));
612         modelVarilla = glm::rotate(modelVarilla, glm::radians(mainWindow.getarticulacion33()), glm::vec3(0.0f, 1.0f, 0.0f));
613         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
614         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelVarilla));
615         Varilla.RenderModel();
616 
617         // Antena Superior
618         color = glm::vec3(0.0, 0.0, 0.0);
619         glm::mat4 modelAntena = modelCuerpoS;
620         modelAntena = glm::translate(modelAntena, glm::vec3(0.0f, 1.06f, 0.0f));
621         modelAntena = glm::rotate(modelAntena, glm::radians(mainWindow.getarticulacion34()), glm::vec3(0.0f, 1.0f, 0.0f));
622         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
623         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelAntena));
624         AntenaSuperior.RenderModel();
```

Capturas de código:





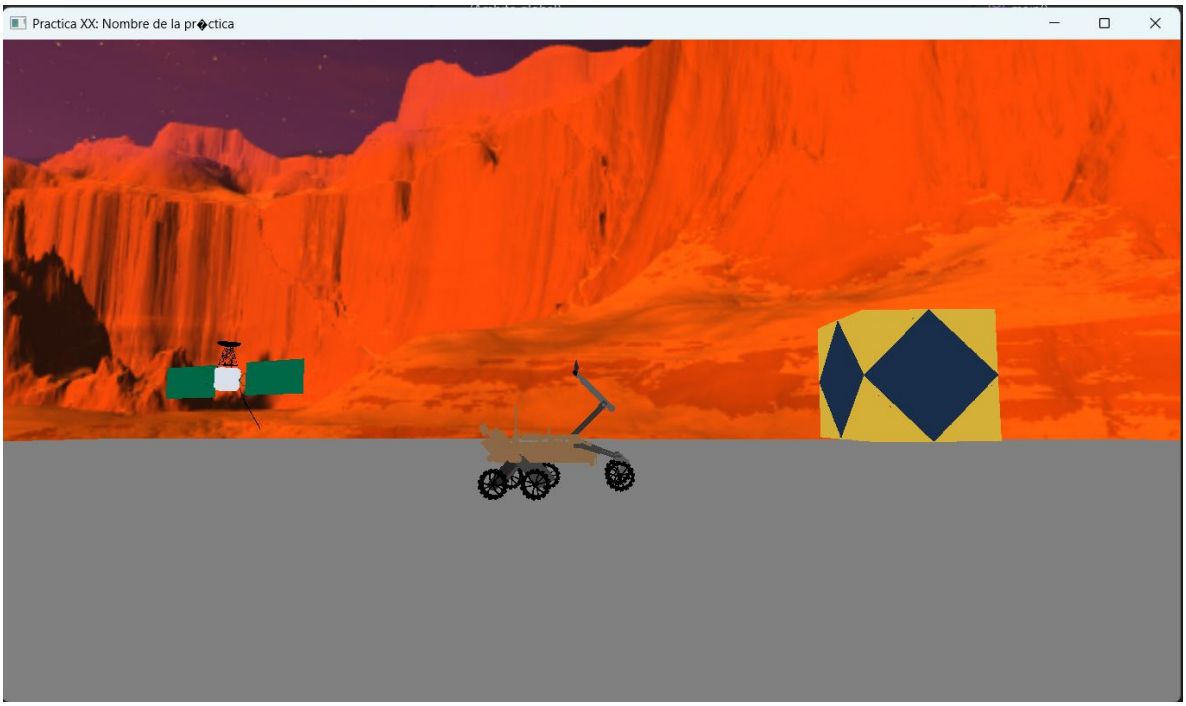
Video de ejecución: https://www.youtube.com/watch?v=jNYHoSSPD3s

Para este ejercicio, las transformaciones de rotación se aplicaron de manera local en los puntos de unión específicos de sus componentes en lugar de girar alrededor de todo el cuerpo central. Tomando como base la matriz del cuerpo principal (modelCuerpoS), se definieron matrices hijas para los elementos móviles como modelPanel1, modelPanel2, modelVarilla y modelAntena a las cuales se les aplicó una traslación local hacia su respectivo punto de articulación antes de ejecutar la rotación controlada por las variables de articulación correspondientes. De esta manera, componentes clave como los paneles solares, la varilla y la antena superior giran de forma independiente sobre sus propios ejes de conexión, simulando con precisión el comportamiento mecánico real de un satélite en el espacio.

Mostrar en la escena al mismo tiempo:

Rover, Holocron, satélite

Capturas de ejecución:



En la ejecución final de la escena gráfica se logró renderizar de manera simultánea e integrada los tres modelos tridimensionales principales: el rover en la parte central, el holocrón hacia la derecha y el satélite flotando a la izquierda sobre el entorno. Esto demuestra el correcto funcionamiento del pipeline gráfico y de las estructuras jerárquicas implementadas, permitiendo que todos los objetos coexistan y se desplacen de forma independiente en un mismo escenario tridimensional.

Video de ejecución: https://www.youtube.com/watch?v=jNYHoSSPD3s

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Para esta práctica no se presentaron mayores problemas más que el factor del tiempo.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.
b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica
c. Conclusión

En conclusión, los ejercicios que conformaron esta práctica presentaron un nivel considerable de complejidad que requirió de un análisis exhaustivo y de una inversión significativa de tiempo para comprender y aplicar la lógica del modelado jerárquico y la manipulación de transformaciones espaciales. Lamentablemente, debido a factores externos y a la alta exigencia de tiempo que demandó la complejidad de la práctica, no me fue posible concluirla al cien por ciento en tiempo y forma, sin embargo, a pesar de todo, considero que fue una muy buena práctica que consolidó de manera significativa los conocimientos respecto a jerarquía, optimización y carga de modelos.

1. Bibliografía en formato APA

Angel, E., & Shreiner, D. (2014). Interactive computer graphics: A top-down approach with shader-based OpenGL (6.ª ed.). Pearson.

Blender Foundation. (s.f.). Blender reference manual. Blender Foundation. Recuperado de https://docs.blender.org/manual/en/latest/

Lengyel, E. (2019). Mathematics for 3D game programming and computer graphics (3.ª ed.). Cengage Learning.

Shirley, P., & Marschner, S. (2009). Fundamentals of computer graphics (3.ª ed.). A K Peters/CRC Press.

Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL programming guide: The official guide to learning OpenGL, version 4.3 (8.ª ed.). Addison-Wesley.

The Khronos Group. (s.f.). OpenGL API Reference Pages. The Khronos Group. Recuperado de https://www.khronos.org/registry/OpenGL-Refpages/